

Developer Functional Specification Document

Project: Automated Skeletal Analysis System

Tech Stack: React (Vite/Tailwind), Node.js (Express), Supabase (PostgreSQL)

1. System Architecture Overview

The system operates on a standard three-tier architecture:

- 1. **Frontend (Client):** A single-page application (SPA) built with React. It manages UI state, form multi-step logic, and data visualization.
- 2. **Backend (API):** An Express.js server that acts as a middleware. It handles business logic, securely communicates with the machine learning prediction model (or Python microservice), and interfaces with the database.
- 3. **Database (Storage):** Supabase PostgreSQL handles persistent storage for case metadata, user profiles, and historical prediction results.

2. Database Schema (Supabase PostgreSQL)

The database should be normalized to separate case metadata from the specific skeletal measurements and model predictions.

Table: users (Managed by Supabase Auth)

(Standard Supabase Auth table handling emails, passwords, and sessions).

Table: cases

Stores the basic information from Step 1 of the analysis.

Column Name	Data Type	Constraints	Description
id	UUID	Primary Key	Unique case identifier.
user_id	UUID	Foreign Key	Links to users.id.

case_number	VARCHAR	Unique	Human-readable ID (e.g., C001).
investigator_name	VARCHAR	Not Null	Name entered in Step 1.
bones_type	VARCHAR	Not Null	e.g., "Skull", "Pelvis".
location	VARCHAR	Not Null	Discovery location.
date_found	DATE	Not Null	Date the remains were found.
analysis_date	DATE	Default: NOW()	Date the record was created.

Table: measurements

Stores the skeletal features from Step 2.

Column Name	Data Type	Constraints	Description
id	UUID	Primary Key	Unique measurement ID.
case_id	UUID	Foreign Key	Links to cases.id (Cascade Delete).

brow_ridge	VARCHAR	Not Null	e.g., "Prominent", "Smooth".
jaw_shape	VARCHAR	Not Null	e.g., "Square", "V-Shape".
nuchal_crest	VARCHAR	Not Null	e.g., "Present", "Absent".
mastoid_size	VARCHAR	Not Null	e.g., "Large", "Small".
cranial_suture	VARCHAR	Not Null	e.g., "Closed", "Open".

Table: predictions

Stores the results generated by the ML model.

Column Name	Data Type	Constraints	Description
id	UUID	Primary Key	Unique prediction ID.
case_id	UUID	Foreign Key	Links to cases.id (Cascade Delete).
predicted_gender	VARCHAR	Not Null	"Male" or "Female".
age_range_min	INT	Not Null	Lower bound of

			predicted age.
age_range_max	INT	Not Null	Upper bound of predicted age.
height_cm	INT	Not Null	Predicted height.
confidence_score	DECIMAL	Not Null	Percentage (e.g., 85.0).

3. Backend API Specifications (Node.js / Express)

All routes should be prefixed with /api/v1 and protected by middleware verifying the Supabase JWT.

3.1 Dashboard Statistics

- **Endpoint:** GET /dashboard/stats
- **Description:** Aggregates data for the top dashboard cards and charts.
- **Response (JSON):**

```
JSON
{
  "total_cases": 158,
  "recent_analysis_count": 89,
  "average_accuracy": 98.8,
  "last_prediction_gender": "Female",
  "age_distribution": [
    {"range": "0-18", "count": 18},
    {"range": "19-30", "count": 68}
  ],
  "gender_distribution": {"male": 60, "female": 40}
}
```

3.2 Run New Analysis

- **Endpoint:** POST /analysis/predict

- **Description:** Receives payload from the frontend Step 1 & 2. The Express server sends this to the ML model, receives the result, saves everything to Supabase, and returns the full case object.

- **Request Body:**

JSON

```
{
  "basic_info": {
    "case_number": "C004",
    "investigator_name": "Chamudi Gayeshika",
    "bones_type": "Skull",
    "location": "Kottawa",
    "date_found": "2026-02-24"
  },
  "measurements": {
    "brow_ridge": "Prominent",
    "jaw_shape": "Square",
    "nuchal_crest": "Present",
    "mastoid_size": "Large",
    "cranial_suture": "Closed"
  }
}
```

- **Response (201 Created):** Returns the newly created case ID, prediction results (Gender, Age, Height, Confidence), and an array of similar_cases.

3.3 Fetch Past Analyses

- **Endpoint:** GET /cases
- **Description:** Fetches historical cases with pagination and search functionality.
- **Query Parameters:**
 - page (int) - Default: 1
 - limit (int) - Default: 10
 - search (string) - Matches against case_number or investigator_name.

- **Response (JSON):**

JSON

```
{
  "data": [ { "case_number": "C001", "investigator_name": "A N Perera", ... } ],
  "meta": { "total_records": 256000, "current_page": 1, "total_pages": 25600 }
}
```

4. Frontend Functional Specifications

4.1 State Management (New Analysis Flow)

Because the "New Analysis" process spans multiple screens (Step 1, Step 2, Step 3), the frontend must hold the form state in memory until the final submission.

- **Implementation:** Use React Context API (AnalysisFormContext) or Zustand/Redux.
- **Behavior:**
 - When a user clicks "Next" on Step 1, update the context state with the Basic Info.
 - When a user clicks "Next" on Step 2, combine the context state with the Measurement data and dispatch the POST /analysis/predict API call.
 - Set a loading state (spinner/skeleton) while waiting for the Express backend to return the ML prediction.
 - Upon success, route to Step 3 and display the returned prediction data.
 - If the user navigates away from the multi-step form before completion, prompt a warning: "Are you sure you want to leave? Your progress will be lost."

4.2 Data Visualization

- Use a charting library (e.g., recharts).
- **Age Distribution Chart:** A standard Bar Chart mapping the age_distribution array from the API.
- **Gender Chart:** A Donut Chart mapping the gender_distribution object. Ensure custom tooltip components are built to display exact numerical values on hover.

4.3 Pagination & Data Tables

- The "Past Analysis" table must not load all 256K entries at once.
- Implement server-side pagination. When the user clicks page "2", the frontend triggers GET /cases?page=2&limit=8.
- The Search bar should utilize a "debounce" hook (e.g., 500ms delay after the user stops typing) before firing the GET /cases?search=... API request to prevent database overloading.

4.4 Report Generation

- On the "Prediction Report" screen, the "Download Report" button should convert the specific DOM elements (Basic Info, Skeletal Features, Prediction Results) into a formatted PDF.
- **Implementation:** Use html2canvas combined with jsPDF to capture the styled React components and export them as an A4 sized PDF document. Ensure dark-mode text colors are inverted or appropriately styled for printing.